

Fractional Teleoperation in ROS 2: Scientific and Algorithmic Description of the Node Implementation

Extender Workspace – fractional_teleoperation_node

April 22, 2026

Abstract

This note documents the mathematical model and the exact implemented algorithm of the ROS 2 standalone node `fractional_teleoperation_node`. The control law is based on a fractional-order relation between joystick input and Cartesian offset, discretized with a finite-memory Grünwald–Letnikov (GL) recursion. The document first introduces the scientific context of fractional analysis, then details dynamic-order adaptation, ramp-based offset scaling, reference drift (first-order and fractional modes), and snap-on-release logic. The goal is scientific consistency with the implementation and operational clarity for tuning and validation.

1 Introduction: Why Fractional Analysis for Teleoperation

Fractional calculus extends differentiation and integration to non-integer order and enables operators with memory, which is attractive in human-in-the-loop systems where command smoothness and motion persistence are important [1, 2, 3, 4].

In classical integer-order teleoperation mappings, response shaping is mostly obtained from static gains and filters. A fractional operator introduces history-dependent behavior directly in the law:

$${}_0^C D_t^\alpha x(t) = f(t), \quad \alpha \in (0, 2), \quad (1)$$

where α controls memory depth and dynamic character. In practice:

- $\alpha \approx 1$ behaves close to an ordinary integrator relation.
- Lower α yields stronger memory and persistent motion trends.
- Variable α enables context-sensitive response (fine motion at low input, faster motion at high input).

The node implementation uses a finite-memory GL recursion for real-time operation, a common pragmatic approximation in control and signal-processing implementations [2, 5].

2 Implemented Control Objective

Let $u_k \in \mathbb{R}^3$ be the sampled joystick input (linear and angular channels treated separately), Δx_k the fractional offset, and $x_{\text{ref},k}$ a drifting reference position. The implemented law is:

$${}_0^C D_t^\alpha \Delta x(t) = K u(t), \quad (2)$$

with reconstructed desired state

$$x_{\text{des},k} = x_{\text{ref},k} + S_k \Delta x_k, \quad (3)$$

where S_k is a time-varying ramp scale (separate maxima for linear and angular channels). The published velocity command is computed from the scaled offset derivative:

$$v_k = \frac{S_k \Delta x_k - S_{k-1} \Delta x_{k-1}}{\Delta t}. \quad (4)$$

3 Discrete Fractional Recursion (GL Approximation)

For memory length L and sample time Δt , coefficients are generated recursively as:

$$c_0 = 1, \quad c_k = c_{k-1} \frac{k-1-\alpha}{k}, \quad k \geq 1. \quad (5)$$

The node update uses:

$$\Delta x_k = (\Delta t)^\alpha K u_k - \sum_{j=1}^{L'} c_j \Delta x_{k-j}, \quad (6)$$

where $L' = \min(L, \text{available history})$. This is applied independently to linear and angular vectors.

Finite-memory implication. The GL operator is truncated to L samples; therefore the implementation is an approximation of ideal long-memory fractional dynamics. Increasing L improves fidelity at increased computational cost.

4 Dynamic Fractional Order α

When enabled, α is adapted from joystick norm $\lambda = \|u_{\text{linear}}\|$:

$$\alpha(\lambda) = \begin{cases} \alpha_{\min}, & \lambda < l_0, \\ \alpha_{\max}, & \lambda > l_{\max}, \\ \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \frac{\lambda - l_0}{l_{\max} - l_0}, & \text{otherwise.} \end{cases} \quad (7)$$

When $|\alpha_{\text{new}} - \alpha_{\text{last}}| > 10^{-3}$, GL coefficients are recomputed.

Implementation caveat (variable order). The history buffers are not reprojected when α changes; only coefficients are refreshed. This is an engineering approximation for real-time operation and should be interpreted accordingly for strict variable-order fractional analysis.

5 Gain Normalization Across Fractional Orders

If normalization is enabled, K is recomputed each cycle using one of two modes:

$$\text{dt mode: } K = v_{\max} (\Delta t)^{1-\alpha}, \quad (8)$$

$$\text{perceptual mode: } K = v_{\max} \frac{\Gamma(\alpha)}{t_{\text{ref}}^{\alpha-1}}. \quad (9)$$

For numerical safety in perceptual mode, a lower bound is used before evaluating $\Gamma(\alpha)$.

6 Offset Ramp Scaling

Joystick activity is detected by $\|u_{\text{linear}}\| > \tau_{\text{active}}$. An active-duration timer T_{active} is incremented while active and reset to zero otherwise. Define

$$r_k = \begin{cases} \min\left(1, \frac{T_{\text{active}}}{T_{\text{ramp}}}\right), & T_{\text{ramp}} > 0, \\ 1, & T_{\text{ramp}} = 0. \end{cases} \quad (10)$$

Then a profile function $\phi(\cdot)$ produces $\phi(r_k)$ and scales become

$$S_k^{\text{lin}} = S_{\max}^{\text{lin}} \phi(r_k), \quad S_k^{\text{ang}} = S_{\max}^{\text{ang}} \phi(r_k). \quad (11)$$

Supported profiles include linear, quadratic, smoothstep, sigmoid, exponential, inverse exponential, tanh, cubic Hermite, and step.

7 Reference Drift and Snap-On-Release

7.1 Reference Drift Gate

Drift is applied only if enabled and

$$\|u_{\text{linear}}\| \geq \tau_{\text{drift}}. \quad (12)$$

7.2 First-Order Reference Drift

In first-order mode:

$$x_{\text{ref},k+1} = x_{\text{ref},k} + \beta \Delta t (x_{\text{des},k} - x_{\text{ref},k}), \quad (13)$$

with clamped blend $\beta \Delta t \in [0, 1]$ in code.

7.3 Fractional Reference Drift

In fractional mode, define error $e_k = x_{\text{des},k} - x_{\text{ref},k}$ and use a second fractional integrator:

$$f_k = \mathcal{F}_{\alpha_r, K_r}(e_k), \quad (14)$$

$$x_{\text{ref},k+1} = x_{\text{ref},k} + \Delta t f_k, \quad (15)$$

where $\mathcal{F}_{\alpha_r, K_r}$ is the same GL-based operator class with parameter mapping

$$(\alpha_r, K_r) = (\text{reference_fractional_alpha}, \text{reference_fractional_gain}). \quad (16)$$

7.4 Snap-On-Release Transition

Let a_k be joystick active state ($a_k \in \{0, 1\}$). If snap is enabled and transition $a_{k-1} = 1 \rightarrow a_k = 0$ occurs, the node executes:

$$x_{\text{ref},k} \leftarrow x_{\text{ref},k} + S_{k-1} \Delta x_{k-}, \quad (17)$$

$$\Delta x_k \leftarrow 0, \quad (18)$$

$$\text{fractional histories} \leftarrow \emptyset, \quad (19)$$

for both linear and angular channels. This makes joystick release recenter the fractional state around the current desired position.

8 Algorithmic Form (Node Update Loop)

At each timer tick:

1. Read latest joystick linear/angular vectors and operation mode.
2. Update dynamic α (if enabled); refresh GL coefficients if threshold exceeded.
3. Update gain K (if adaptive gain normalization enabled).
4. Save previous scaled offsets for derivative computation.
5. Apply GL fractional integration to linear and angular channels to update Δx_k .
6. Update ramp timer from activity condition and compute profile-based scales.
7. Form scaled offsets and desired state $x_{\text{des},k} = x_{\text{ref},k} + S_k \Delta x_k$.
8. If snap condition is met (active-to-inactive transition), shift reference by previous scaled offset, clear fractional offsets and histories.
9. If reference drift gate is open, update reference with first-order or fractional mode.

10. Compute velocities from scaled-offset derivative.
11. Transform to selected frame (base or end-effector), apply translation/rotation mode gating, then apply output velocity scale.
12. Publish velocity command and optional markers.

9 Runtime Parameter Set Used in Current Configuration

The active YAML in this workspace uses the following key values:

Parameter	Value
α	0.5
K (manual, no normalization)	1.0
L (memory length)	50
Δt	0.01 s
Dynamic alpha	enabled, $\alpha_{\min} = 0.1$, $\alpha_{\max} = 1.0$, $l_0 = 0.5$, $l_{\max} = 1.0$
Ramp scaling	enabled via $S_{\max}^{\text{lin}} = 2.0$, $S_{\max}^{\text{ang}} = 1.0$, $T_{\text{ramp}} = 2.5$ s, profile exponential
Snap on release	enabled
Reference drift	enabled, threshold $\tau_{\text{drift}} = 0.5$, mode fractional, $\alpha_r = 0.5$, $K_r = 0.5$

10 Reproducible Analysis Outcomes (Analysis Folder)

An executable analysis pipeline is provided in the package analysis folder:

- model and primitives: `analysis/fractional_node_model.py`
- runner: `analysis/run_analysis.py`
- baseline outputs: `analysis/results/`

The baseline command is:

$$\text{/bin/python3 run_analysis.py} \tag{20}$$

executed from `src/controllers/fractional_teleoperation/analysis`.

Generated outputs include:

- linear fixed- α stability table and spectral-radius plot,
- controllability rank and finite-horizon Gramian table,
- bounded-input (ISS-style) response table,
- hybrid threshold sweep table and sensitivity plot,
- α -memory sweep table and heatmap.

For the current baseline:

- Spectral radius increases with α and reaches 1.0 at $\alpha = 1.0$ (marginal in this discrete linear check), while remaining below 1 for $\alpha \in [0.1, 0.9]$.
- Controllability rank equals state dimension (49) on the tested fixed- α augmented models, while Gramian conditioning degrades with increasing α .

- In bounded random-input tests ($\|u\|_\infty \leq 1$), peak velocity magnitude decreases strongly with α (about 357 at $\alpha = 0.1$ to about 3.29 at $\alpha = 1.0$), while reference excursion increases.
- In hybrid threshold sweeps, increasing `joystick_active_threshold` from 0.005 to 0.05 increases observed transition count (99 to 274) and snap events (49 to 137), with mean dwell time decreasing (12.0 to 4.36 steps).
- In the α -memory sweep, peak velocity is primarily α -driven (weak dependence on memory length), but reference excursion grows with memory length.

These results are implementation-faithful computational checks, not formal global proofs for the full variable-order hybrid system.

11 Scientific and Engineering Notes

- The finite-memory GL update is a practical approximation; sensitivity to memory length should be evaluated experimentally.
- With variable α , coefficient refresh without history re-identification is computationally efficient but not a strict variable-order discretization.
- Perceptual gain normalization can produce large gains at very low α because of $\Gamma(\alpha)$ behavior; practical lower bounds and safety saturation remain essential.
- Drift and snap combine to reduce long-term offset accumulation and improve joystick recentering ergonomics.

12 Scope Boundary

This document targets the standalone node implementation only. The ROS 2 controller plugin in the same package shares core fractional routines but does not include all node-specific behavioral blocks (notably the same drift/snap/ramp orchestration).

References

- [1] K. B. Oldham and J. Spanier, *The Fractional Calculus*. Academic Press, 1974.
- [2] I. Podlubny, *Fractional Differential Equations*. Academic Press, 1999.
- [3] C. A. Monje, Y. Chen, B. M. Vinagre, D. Xue, and V. Feliu, *Fractional-Order Systems and Controls: Fundamentals and Applications*. Springer, 2010.
- [4] K. Diethelm, *The Analysis of Fractional Differential Equations*. Springer, 2010.
- [5] A. Oustaloup, F. Levron, B. Mathieu, and F. M. Nanot, “Frequency-band complex noninteger differentiator: characterization and synthesis,” *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, vol. 47, no. 1, pp. 25–39, 2000.
- [6] D. Valério and J. Sá da Costa, *An Introduction to Fractional Control*. Institution of Engineering and Technology, 2013.